

The pyplot command

Matplotlib plotting in ngspice — complete reference

Ngspice + OpenVAF Enhancements project

July 2026

Contents

The pyplot command	1
1. Requirements	2
2. The basics	2
3. Interactive window vs. image file	3
4. Appearance	3
5. Axes, limits and labels	4
6. Histograms — -hist	5
7. Contour maps — -contour	6
8. Smith charts — -smith	6
9. Eye diagrams — -eye	7
10. Spectra — -fft	7
11. Complex-aware AC — -bode, -nyquist, -polar	8
12. Comparing runs and reading values	9
13. What gets written	10
14. Worked example — one deck, several views	10
15. Quick reference	11
16. Troubleshooting	11
See also	12

The **pyplot** command

pyplot renders ngspice vectors with matplotlib. It is the Python counterpart of the built-in gnuplot command: it writes a self-contained Python script plus its data next to your deck and runs them, either opening an interactive window or writing an image file.

```
pyplot [file] plotargs
pyplot [file] -eye      <expr> -ui <T> [-tstart t] [-threshold v] [-window f]
pyplot [file] -hist     <vec> [vec ...]
pyplot [file] -contour <z> <x> <y>
pyplot [file] -smith    <vec> [vec ...]
pyplot [file] -fft       <vec> [vec ...]
pyplot [file] -bode      <vec> [vec ...]
pyplot [file] -nyquist  <vec> [vec ...]
pyplot [file] -polar    <vec> [vec ...]
```

Everything below was checked against the shipped implementation (src/frontend/com_pyplot.c, src/frontend/plotting/pyplot.c); every example in this document was executed before being written down.

1. Requirements

pyplot shells out to a Python interpreter that must have matplotlib importable.

```
python3 -c "import matplotlib; print(matplotlib.__version__)"
```

If your interpreter is not python3, point ngspice at it:

```
set pyplot_python=/opt/homebrew/bin/python3.12
```

2. The basics

The simplest form takes one or more plot expressions — anything plot accepts:

```
* RC lowpass, plotted with matplotlib
v1 in 0 dc 0 ac 1 sin(0 1 1k)
r1 in out 1k
c1 out 0 159.155n
.control
tran 10u 3m
pyplot v(in) v(out)
.endc
.end
```

2.1 The file name is optional

The first word is treated as an output base name only when it is not itself a plot expression — that is, when it contains no (*and* does not name an existing vector. Otherwise all words are plot arguments and the base name defaults to pyplot.

```
pyplot v(out)           $ no name -> base name "pyplot"
pyplot rcplot v(out)     $ "rcplot" is not a vector -> it IS the base name
pyplot b                 $ "b" IS a vector          -> it is PLOTTED, not a file name
```

That last line is the rule worth remembering: if a node happens to be called rcplot, pyplot rcplot plots it rather than naming a file after it.

2.2 Successive unnamed plots get distinct names

A per-session counter keeps default-named plots from overwriting each other — important in window mode, where viewers run in the background and would otherwise race for the same files. The counter is shared across all modes:

```
pyplot v(a)              -> pyplot.py    / pyplot.data
pyplot v(b)              -> pyplot-2.py / pyplot-2.data
pyplot -contour z x y     -> contour-3.py
pyplot v(c)              -> pyplot-4.py
```

The mode supplies the stem (pyplot, eye, contour, smith); the counter supplies the suffix.

2.3 Where the files go

With a bare base name, artifacts are written next to the circuit file, so a self-contained deck folder collects its own plots. Give a path of your own and it is respected. The special names temp / tmp write to a

temporary file instead:

```
pyplot temp v(out)
```

3. Interactive window vs. image file

By default `pyplot` opens an interactive matplotlib window. Setting `pyplot_terminal` switches to hard-copy:

set pyplot_terminal=	Result
(unset)	interactive window
png or png/quit	writes <name>.png
svg or svg/quit	writes <name>.svg
pdf or pdf/quit	writes <name>.pdf

* three formats from the same data

```
.control
tran 10u 3m
set pyplot_terminal=png
pyplot exppng v(in) v(out)
set pyplot_terminal=svg
pyplot expsvg v(in) v(out)
set pyplot_terminal=pdf
pyplot exppdf v(in) v(out)
.endc
```

svg and pdf are vector formats — use them for publication figures.

On a headless machine (CI, a remote shell with no display) also pin the matplotlib backend, or matplotlib may fail trying to find a GUI:

```
set pyplot_backend=Agg
```

4. Appearance

All of these are ordinary ngspice variables — set them before the `pyplot` call, unset them to return to the default.

Variable	Meaning	Default
pyplot_terminal	png / svg / pdf (see above)	interactive window
pyplot_backend	matplotlib backend, e.g. Agg	matplotlib's choice
pyplot_python	interpreter to run	python3
pyplot_style	matplotlib style sheet; dark aliases dark_background	matplotlib default
pyplot_figsize	figure size in inches, W, H (also WxH)	matplotlib default
pyplot_linewidth	line width in points, applied to every trace	matplotlib default
pyplot_subplots	traces per stacked panel; 0/unset = one axis	0
pyplot_pointstyle	set to markers to draw point markers	lines only
pyplot_markers	a cycling marker on the line, per trace	lines only
pyplot_grid	on / off / x / y — override the default	axis-dependent
pyplot_legend	off, or a matplotlib location (upper_right, best)	shown

Variable	Meaning	Default
pyplot_axhline	comma list of y values → horizontal reference lines	none
pyplot_axvline	comma list of x values → vertical reference lines	none
pyplot_dpi	image resolution for a hardcopy	100
pyplot_transparent	transparent figure background for a hardcopy	opaque
pyplot_cursor	hover crosshair in an interactive window	off

```
* a styled figure
.control
tran 10u 3m
set pyplot_terminal=png
set pyplot_style=dark
set pyplot_linewidth=2.5
set pyplot_figsize=8,4
pyplot styled v(out)
unset pyplot_style
.endc
```

4.1 Stacked panels

pyplot_subplots=N puts N traces per panel and stacks the panels on a shared x-axis — convenient when traces have wildly different magnitudes:

```
set pyplot_subplots=1      $ one trace per panel
pyplot panels v(in) v(out) $ -> two stacked panels

set pyplot_subplots=2      $ two traces per panel
pyplot p2 v(in) v(a) v(b) v(c) $ -> two panels of two
```

4.2 Grid, legend, markers and reference lines

```
set pyplot_grid=off        $ or on / x / y
set pyplot_legend=upper_right $ or off; use the UNDERSCORE form (see below)
set pyplot_markers         $ a cycling marker on each line
set pyplot_axhline=0.5,-0.5 $ horizontal reference lines
set pyplot_axvline=1k,10k  $ vertical lines (SI suffixes accepted)
set pyplot_dpi=150         $ image resolution
set pyplot_transparent     $ transparent background
pyplot fig db(v(out))
```

Two points worth knowing:

- Legend locations contain a space (upper right), but set keeps only the first word. Use the underscore form — set pyplot_legend=upper_right — and the renderer converts it back.
- **pyplot_markers** vs **pointstyle=markers**. pointstyle=markers draws markers with *no* line; pyplot_markers keeps the line and adds a marker whose shape cycles per trace (o s ^ D v * P X), so overlaid traces are distinguishable in print or greyscale.

5. Axes, limits and labels

pyplot goes through the same argument parser as plot, so the usual plot keywords work:

Keyword	Effect
xlog, ylog, loglog	logarithmic axes
xlimit lo hi	x-axis range
ylimit lo hi	y-axis range
title <text>	figure title
xlabel <text>, ylabel <text>	axis labels
vs <vec>	plot against another vector instead of the default scale

```
* Bode magnitude and phase
.control
ac dec 20 10 1e6
set pyplot_terminal=png
pyplot acmag db(v(out)) xlog title "Magnitude" ylabel "dB"
pyplot acph ph(v(out)) xlog title "Phase"      ylabel "rad"
.endc
```

A parametric (x-y) plot uses vs:

```
pyplot iv i(v1) vs v(a) title "I-V curve"
```

6. Histograms — **-hist**

-hist renders the value distribution of each listed vector instead of its waveform. It is a render mode over the normal signal list, so naming and file options behave exactly as above.

```
* distribution of a sine and a ramp
v1 a 0 dc 0 sin(0 1 1k)
r1 a 0 1k
.control
tran 10u 5m
set pyplot_terminal=png
let sine = v(a)
let ramp = time*200
pyplot h1 -hist sine ramp
.endc
.end
```

Variable	Meaning	Default
pyplot_hist_bins	number of bins	matplotlib auto
pyplot_hist_ density	boolean — normalise to a probability density	counts

```
set pyplot_hist_bins=40
set pyplot_hist_density
pyplot h2 -hist sine
unset pyplot_hist_density
```

A sine's histogram is the classic bathtub shape (most time spent near the peaks); a linear ramp's is flat. That makes -hist a quick sanity check on Monte-Carlo or noise output.

7. Contour maps — **-contour**

-contour takes exactly three vectors — **z, x, y** — and draws **z** as a filled contour map over the (**x, y**) plane. The three must be the same length: each index contributes one (**x, y, z**) triple.

```
* z = sin(x/3)*cos(y/3) on a 20 x 20 grid
v1 a 0 dc 0
r1 a 0 1k
.control
op
set pyplot_terminal=png
let n = 400
let x = vector(n) % 20
let y = floor(vector(n) / 20)
let z = sin(x/3) * cos(y/3)
pyplot c1 -contour z x y
.endc
.end
```

Variable	Meaning	Default
pyplot_contour_levels	number of contour levels	matplotlib's choice
pyplot_contour_cmap	colormap name, e.g. turbo, plasma	viridis
pyplot_contour_lines	boolean — overlay contour lines on the fill	fill only

```
set pyplot_contour_levels=24
set pyplot_contour_cmap=turbo
set pyplot_contour_lines
pyplot c2 -contour z x y
```

Typical uses are two-parameter sweeps: efficiency over (bias, load), delay over (voltage, temperature), or a load-pull surface.

8. Smith charts — **-smith**

-smith plots complex reflection coefficients on a Smith chart. The natural inputs are the **S_{i_j}** vectors produced by an **sp** analysis.

```
* S-parameters of an L-match, on a Smith chart
v1 in 0 dc 0 ac 1 portnum 1 z0 50
r1 in mid 25
l1 mid out 10n
c1 out 0 2p
v2 out 0 dc 0 ac 0 portnum 2 z0 50
.control
sp lin 101 100meg 5g
set pyplot_terminal=png
pyplot sm1 -smith S_1_1
pyplot sm2 -smith S_1_1 S_2_2
.endc
.end
```

Any complex vector works, so a matching trajectory computed with `let` can be drawn the same way:

```
pyplot smith_match -smith gmatch
```

9. Eye diagrams — **-eye**

`-eye` is different from the other three flags: it is not a render mode but a separate analysis. `pyplot ... -eye` runs the `eye` command — which folds the waveform onto one unit interval and computes the eye metrics — and then renders the result.

```
pyplot [name] -eye <expr> -ui <T> [-tstart t0] [-threshold vth] [-window frac]
```

Sub-option	Meaning
<code>-ui <T></code>	unit interval (bit period) — required
<code>-tstart <t></code>	ignore everything before <code>t</code> (skip start-up transients)
<code>-threshold</code>	decision threshold; default is the midpoint of the measured levels
<code><v></code>	
<code>-window</code>	fraction of the UI used for the eye-opening measurement
<code><frac></code>	

* eye diagram of a PRBS through a lossy interconnect

```
Vtx tx 0 PWL(...)
Rc tx rx 250
Cc rx 0 1p
.tran 1p 30n
.control
run
set pyplot_terminal=png
pyplot eyefig -eye v(rx) -ui 0.5n -tstart 3n
.endc
.end
```

Alongside the figure, the eye analysis prints its metrics and leaves the folded waveform in vectors you can post-process:

```
eye: UI 5e-10 s, 21 crossings, levels [0.004027, 0.9872] (amp 0.9832), threshold 0.4956
  eye height : 0.5727 (58.3% of amplitude)
  eye width  : 4.637e-10 s (92.7% of UI); at BER 1e-12: 2.898e-10 s
  folded eye in 'eye_wave' vs 'eye_t' (plot eye_wave vs eye_t)
```

Because `eye_wave` and `eye_t` are ordinary vectors, you can plot them yourself:

```
pyplot manual eye_wave vs eye_t
```

10. Spectra — **-fft**

`-fft` plots the one-sided amplitude spectrum of each signal — `fft-then-plot` in one command.

* spectrum of a transient signal

```
v1 a 0 dc 0 sin(0 2 1k)
r1 a 0 1k
.control
tran 5u 20m
```

```

let sig = v(a)
set pyplot_terminal=png
pyplot spectrum -fft sig
.endc
.end

```

Transient data is adaptively sampled, so the generated script resamples each signal onto a uniform grid before the transform (a raw FFT over non-uniform samples would be wrong). The magnitude is scaled so a pure tone reads back its amplitude at its frequency — a $2 \cdot \sin(2\pi \cdot 1\text{kHz} \cdot t)$ tone peaks at 2.0.

Variable	Meaning	Default
pyplot_fft_ window	hann / hamming / blackman / rect	hann
pyplot_fft_db	plot $20 \cdot \log_{10}$ magnitude	linear
pyplot_fft_ points	resample / FFT length	next power of two $\geq$ len
pyplot_fft_logf	log frequency axis (drops the DC bin)	linear

```

set pyplot_fft_window=blackman
set pyplot_fft_db
set pyplot_fft_logf
pyplot spectrum -fft sig

```

Use **pyplot_fft_logf**, not **xlog**. The command's **xlog** makes ngspice validate the *time* scale (which includes $t = 0$) and abort before the FFT runs; **pyplot_fft_logf** sets the log axis on the frequency data in Python and drops the DC bin.

11. Complex-aware AC — **-bode**, **-nyquist**, **-polar**

An ordinary `pyplot v(out)` on AC data keeps only the real part: at the -3 dB point of an RC low-pass it shows 0.5, not the magnitude 0.7071. (It is the SPICE convention — `mag()`/`db()`/`ph()` are the explicit escapes.) These three modes use the full complex value instead.

```

* frequency response, three views
v1 in 0 dc 0 ac 1
r1 in out 1591.55
c1 out 0 100n
.control
ac dec 30 10 1e6
set pyplot_terminal=png
pyplot resp -bode    v(out)    $ magnitude(dB)/phase(deg) vs log-frequency
pyplot resp -nyquist v(out)    $ imag vs real
pyplot resp -polar   v(out)    $ magnitude at phase, polar projection
.endc
.end

```

Mode	View
-bode	stacked $20 \cdot \log_{10} H $ (dB) and unwrapped phase (deg) vs a log frequency axis
-	imag(H) vs real(H), equal aspect, real/imag axes marked
nyquist	
-polar	$ H $ at angle(H) on a polar projection

Each accepts several vectors (overlaid) and honours the shared `pyplot_*` settings. For the RC low-pass above, the Bode plot reads exactly -3.01 dB and -45° at f_c — because the imaginary part is preserved.

12. Comparing runs and reading values

12.1 Overlay several runs

ngspice keeps each run in its own plot (`tran1`, `tran2`, ...); reference them with `plotname.vector` to overlay:

```
tran 5u 3m          $ -> tran1
alter c1 c=400n
tran 5u 3m          $ -> tran2
pyplot compare tran1.v(out) tran2.v(out)
```

Runs of different lengths overlay correctly — each trace keeps all of its own samples.

12.2 Interactive crosshair and toolbar

In an interactive window (no `pyplot_terminal`), matplotlib's own toolbar already gives pan, zoom and save-image. The value cursor is off by default; `pyplot_cursor` is the one switch that turns it on:

```
set pyplot_cursor
pyplot v(out)
```

That draws matplotlib's built-in crosshair — no extra Python package — and is ignored for a hardcopy, where there is no mouse.

If you have the [mplcursors](#) package installed, add `pyplot_mplcursors` to switch the backend to data cursors that snap to a trace and show its value on hover:

```
set pyplot_python=python3.13    $ an interpreter that has mplcursors
set pyplot_cursor                $ the cursor must be on ...
set pyplot_mplcursors            $ ... and this selects the mplcursors backend
pyplot v(out)
```

The gating is a simple master switch plus a backend selector:

<code>pyplot_cursor</code>	<code>pyplot_mplcursors</code>	Result
unset	<i>(either)</i>	no cursor (the default)
set	unset	built-in crosshair
set	set	<code>mplcursors</code> data cursors

`pyplot_mplcursors` on its own — with `pyplot_cursor` unset — does nothing. When the cursor is on and `mplcursors` is selected, the generated script still falls back to the built-in crosshair if `mplcursors` cannot be imported where it runs, so a deck stays portable. `mplcursors` must be importable by the interpreter named by `pyplot_python`.

12.3 The data is already exported

Every `pyplot` writes `<name>.data` (the plotted columns) next to `<name>.py`. That file is the export — load it anywhere:

```
import numpy as np
d = np.loadtxt("compare.data")
```

13. What gets written

For base name NAME, pyplot produces:

File	Contents
NAME.py	a standalone matplotlib script
NAME.data	the plotted data, one column per vector
NAME.png / .svg / .pdf	the image, when pyplot_terminal is set

NAME.py is *self-contained*: it can be edited and re-run outside ngspice, which is the easiest way to fine-tune a figure for a paper without re-running the simulation.

```
python3 rcplot.py
```

14. Worked example — one deck, several views

```
* pyplot tour: transient, spectrum, panels and a histogram
```

```
v1 in 0 dc 0 ac 1 sin(0 1 1k)
```

```
r1 in out 1k
```

```
c1 out 0 159.155n
```

```
.control
```

```
set pyplot_terminal=png
```

```
set pyplot_backend=Agg
```

```
set pyplot_figsize=8,4.5
```

```
* 1. time domain, two traces on one axis
```

```
tran 10u 3m
```

```
pyplot tour_tran v(in) v(out) title "RC step response"
```

```
* 2. the same data, one trace per panel
```

```
set pyplot_subplots=1
```

```
pyplot tour_panels v(in) v(out)
```

```
unset pyplot_subplots
```

```
* 3. distribution of the output
```

```
let vo = v(out)
```

```
set pyplot_hist_bins=50
```

```
pyplot tour_hist -hist vo
```

```
unset pyplot_hist_bins
```

```
* 4. frequency domain, log x, dark style
```

```
ac dec 20 10 1e6
```

```
set pyplot_style=dark
```

```
pyplot tour_bode db(v(out)) xlog title "Bode magnitude" ylabel "dB"
```

```
unset pyplot_style
```

```
.endc
```

```
.end
```

15. Quick reference

Modes

Form	Renders
<code>pyplot [f] <exprs></code>	ordinary traces
<code>pyplot [f] -hist <vecs></code>	value histograms
<code>pyplot [f] -contour <z> <x> <y></code>	filled contour map
<code>pyplot [f] -smith <vecs></code>	Smith chart
<code>pyplot [f] -eye <expr> -ui <T></code>	eye diagram (runs the eye analysis)
<code>pyplot [f] -fft <vecs></code>	amplitude spectrum
<code>pyplot [f] -bode <vecs></code>	magnitude(dB)/phase(deg) vs log-f
<code>pyplot [f] -nyquist <vecs></code>	imag vs real
<code>pyplot [f] -polar <vecs></code>	magnitude at phase (polar)

Variables

Variable	Type	Applies to
<code>pyplot_terminal</code>	png/svg/pdf	all
<code>pyplot_backend</code>	string	all
<code>pyplot_python</code>	string	all
<code>pyplot_style</code>	string	all
<code>pyplot_figsize</code>	W,H	all
<code>pyplot_linewidth</code>	real	line plots
<code>pyplot_subplots</code>	integer	line plots
<code>pyplot_markers</code>	boolean	line plots
<code>pointstyle</code>	markers	line plots
<code>pyplot_grid</code>	on/off/x/y	line plots
<code>pyplot_legend</code>	off/location	line plots
<code>pyplot_axhline / pyplot_axvline</code>	value list	line plots
<code>pyplot_dpi</code>	integer	hardcopy
<code>pyplot_transparent</code>	boolean	hardcopy
<code>pyplot_cursor</code>	boolean	window
<code>pyplot_mplcursors</code>	boolean	window
<code>pyplot_hist_bins</code>	integer	-hist
<code>pyplot_hist_density</code>	boolean	-hist
<code>pyplot_fft_window</code>	string	-fft
<code>pyplot_fft_db / pyplot_fft_logf</code>	boolean	-fft
<code>pyplot_fft_points</code>	integer	-fft
<code>pyplot_contour_levels</code>	integer	-contour
<code>pyplot_contour_cmap</code>	string	-contour
<code>pyplot_contour_lines</code>	boolean	-contour

16. Troubleshooting

Nothing appears, no error. `pyplot_terminal` is unset, so a window was opened — on a headless machine there is nowhere to draw it. Set `pyplot_terminal=png` and `pyplot_backend=Agg`.

ModuleNotFoundError: matplotlib. The interpreter ngspice used lacks matplotlib. Check which one it is and override with `set pyplot_python=....`

Two plots show the same thing. Both were given the same explicit base name, so the second overwrote the first. Omit the name and let the auto-counter do it, or give distinct names.

-contour complains or looks wrong. It needs exactly three vectors, $z \times y$, all of the same length.

A file name got plotted as a vector. The first word names an existing vector, so it was treated as data. Rename the output or the node.

See also

- `examples/pyplot_examples/` — the base command, naming and export
- `examples/pylohist_examples/` — `-hist`
- `examples/pyplotcontour_examples/` — `-contour`
- `examples/pyplotsmith_examples/` — `-smith`
- `examples/pyplotmore_examples/` — appearance controls, `-fft`, `-bode/-nyquist/-polar`, `overlay`
- Enhancement write-ups: 94, 95, 98, 99, 182, 183, 208, 217, 218, 254, 296, 297, 298, 299, 300, 301